

Organización de Computadoras

Arquitecturas Microprogramadas - Subrutinas

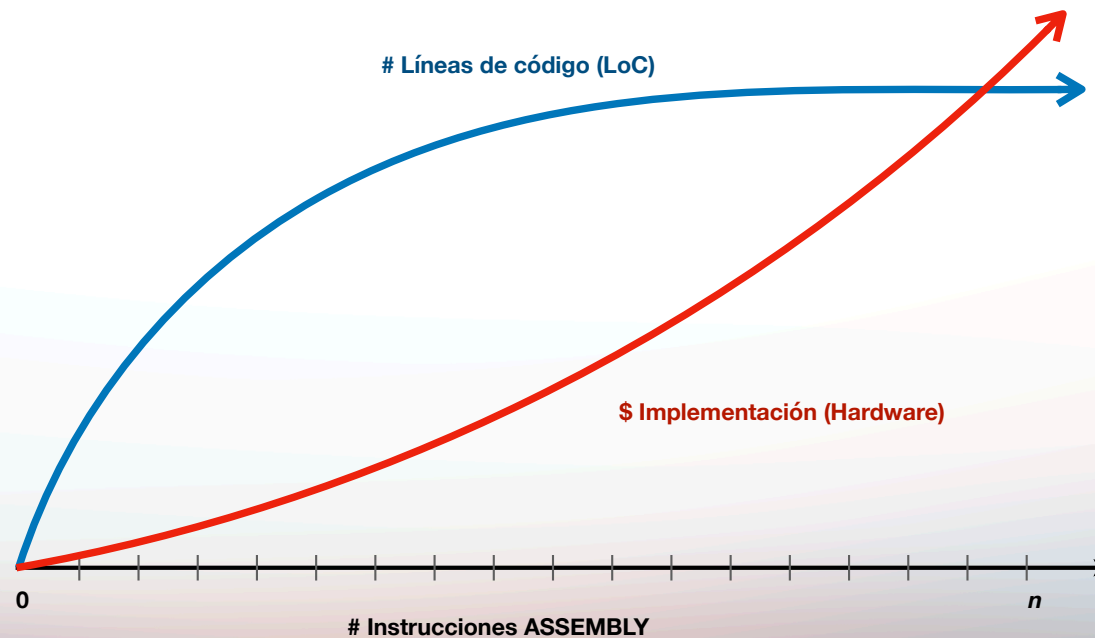
Departamento de Computación - UNRC

El camino a recorrer

- Un poco de Historia y Sistemas Numéricos
- Introducción a la Electrónica
- Representación de Información
- Cómo computar utilizando la electricidad
- Evolución y funcionamiento abstracto de una computadora
- **Arquitecturas microprogramadas**
- Assembly X86
- Eficiencia
 - Pipelines
 - Memoria Caché

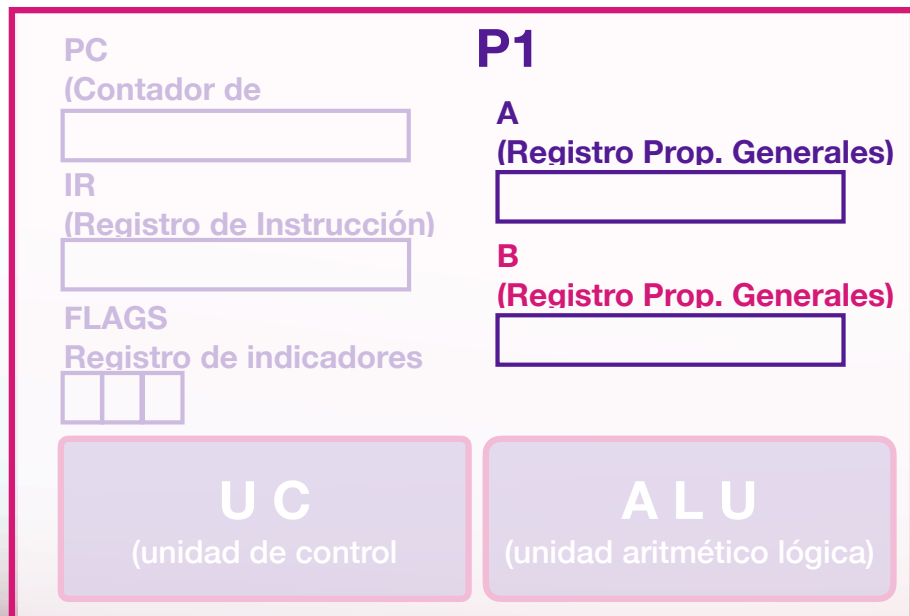
CPUSim - **Complex** Ints. Set Computer vs **Reduced** Ints. Set Computer

Relación entre Conjunto de Instrucciones, Costo de Hardware y tamaños de programas.



CPUSim - P1 (v2) Agregando un registro más y moviendo información entre ellos

Si bien con un solo registro de propósito general se puede computar todo lo computable, disponer de más registros de este tipo permite, en general, reducir la cantidad de instrucciones en lenguaje ensamblador (ASSEMBLY) necesarias para implementar los programas.



Type of Module: Register			
name	width	initial value	read-only
A	16	0	☐
B	16	0	☐

CPUSim - P1 (v2) Agregando un registro más y moviendo información entre ellos

Si bien con un solo registro de propósito general se puede computar todo lo computable, disponer de más registros de este tipo permite, en general, reducir la cantidad de instrucciones en lenguaje ensamblador (ASSEMBLY) necesarias

Como usaremos más adelante, asumimos el DESTINO como el primer operando (similar a una asignación)

The screenshot displays the CPUSim - P1 (v2) interface, which is divided into several sections:

- Left Panel (CPU Architecture):** Contains fields for the Instruction Register (IR), Flags, and the Program Counter (PC). Below these is a box labeled "UC (unidad de control)".
- Register Section:** Shows two general-purpose registers, A and B, both 16 bits wide and initialized to 0. A circular arrow indicates data movement between them.
- Assembly Editor:** Displays the instruction "A->B" with its opcode "0xA". The instruction format is shown as a bit field: 4 bits for the opcode, 1 bit for the register number, and 10 bits for the destination register.
- Execution Log:** Shows the sequence of operations: "Si es Reg A", "<IR[4]==0>(2)", "A->B", "End", "B->A", and "End".
- REGID Table:** A table showing the register values for the current instruction. It has columns for Name and Value. The values are: B: 1, A: 0.

A red arrow points from the "REGID" table to the "R...P..." field in the assembly editor, indicating the register used for the operation.

CPUSim - P1 (v2) Adecuando otras Instrucciones a 2 registros

Para ser brindar un uso más simple podemos incorporar como operando qué registro es origen o destino de las Instrucciones que ya implementamos:

In <R>



Execute sequence

<IR[4]==0>(2)

IO->B

End

IO->A

End

Out <R>



Execute sequence

<IR[4]==0>(2)

B->IO

End

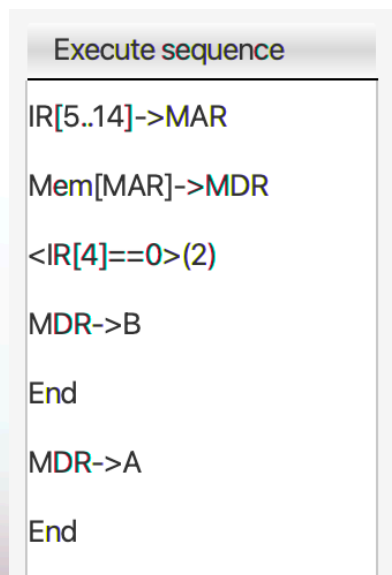
A->IO

End

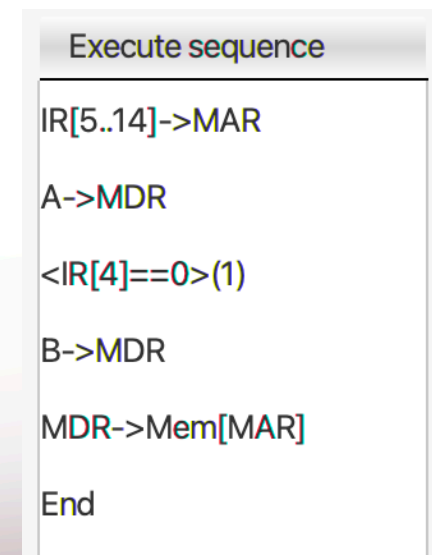
CPUSim - P1 (v2) Adecuando otras Instrucciones a 2 registros

Para ser brindar un uso más simple podemos incorporar como operando qué registro es origen o destino de las Instrucciones que ya implementamos:

Load <R> <Dir>



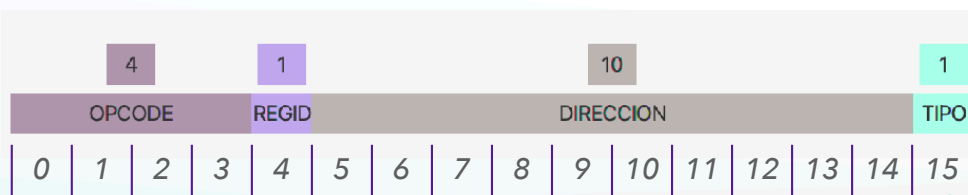
Save <R> <Dir>



CPUSim - P1 (v2) Adecuando otras Instrucciones a 2 registros

Para ser brindar un uso más simple podemos incorporar como operando qué registro es origen o destino de las Instrucciones que ya implementamos:

ADD <R> <Dir> <I/D>



Inmediato

+ Valor Positivo

A

A

Execute sequence

//Analizar TIPO

<IR[15]==1>(8)

IR[5..14]->MAR

Mem[MAR]->MDR

<IR[4]==0>(2)

MDR+B->B

End

MDR+A->A

End

Determinar Registros

//Inmediato

RESET-MDR-0

<IR[5]==0>(1)

RESET-MDR-1

IR[5..14]->MDR

<IR[4]==0>(2)

MDR+B->B

End

MDR+A->A

End

Determinar Registros

CPUSim - P1 (v2) Ejemplo de uso

```
1 ; Programa que intercambia el valor de 2 registros
2 ; Luego suma en A un valor positivo y en B uno negativo
3
4     In A ;ingreso valor en A
5     In B ;ingreso valor en A
6     ; intercambio los valores
7     Save A aux ;guardo el valor de A en aux
8     Move A B ;copio el valor de B en A
9     Load B aux ;guardo el valor de aux B
10    ; Sumo valores inmediatos
11    Add A 10 I
12    Add B -10 I
13    Out A ;muestro valor en A
14    Out B ;muestro valor en B
15    Stop
16
17 aux: .data 2 0; variable auxiliar
```

EXECUTING...

Enter Inputs, the first of which must be an Integer: 3

Enter Inputs, the first of which must be an Integer: 7

Output: 17

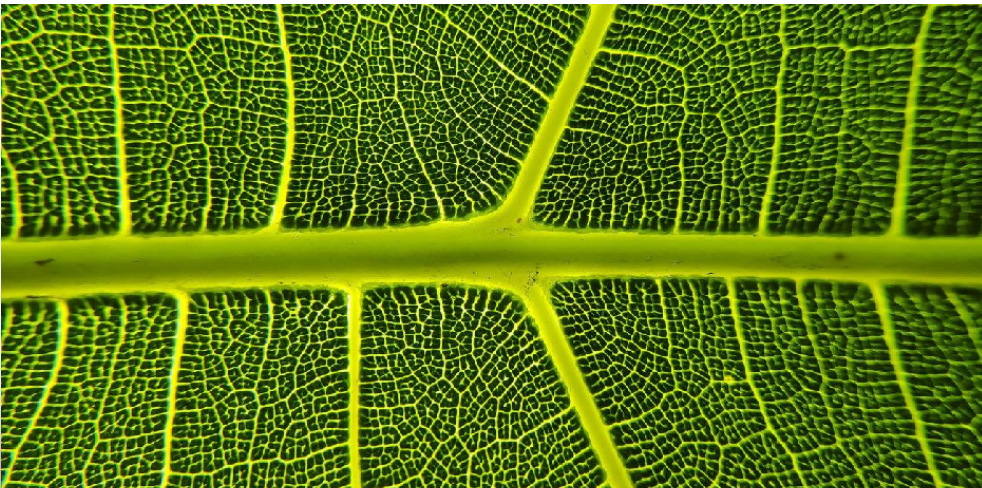
Output: -7

EXECUTION HALTED NORMALLY due to the setting of the bit(s): [HALT]

Hacia la posibilidad de modularizar código (ASSEMBLY)



Comenzaremos por entender el concepto y cómo utilizarlo desde la perspectiva de un usuario de lenguaje ensamblador (ASSEMBLY).



Luego implementaremos en CPU-Sim mejorando nuestro procesador para soportar su utilización.

Hacia la posibilidad de modularizar código (ASSEMBLY)



Comenzaremos por entender el concepto y cómo utilizarlo desde la perspectiva de un usuario de lenguaje ensamblador (ASSEMBLY) suponiendo una versión mejorada de nuestro procesador P1.

Hacia la posibilidad de modularizar código

Una **subrutina** es una porción de código **independiente** que realiza una tarea específica dentro de un programa más grande. Las subrutinas se utilizan para **modularizar** el código y mejorar la **organización**, permitiendo **reutilizar** funcionalidades en diferentes partes del programa.

Programa

```
In A
In B
; intercambio los valores
Save A aux
Move A B
Load B aux
; -----
Out A
Out B
Load A D1
Load B D2
; intercambio los valores
Save A aux
Move A B
Load B aux
; -----
Save A D1
Save B D2
Stop
```

Hacia la posibilidad de modularizar código

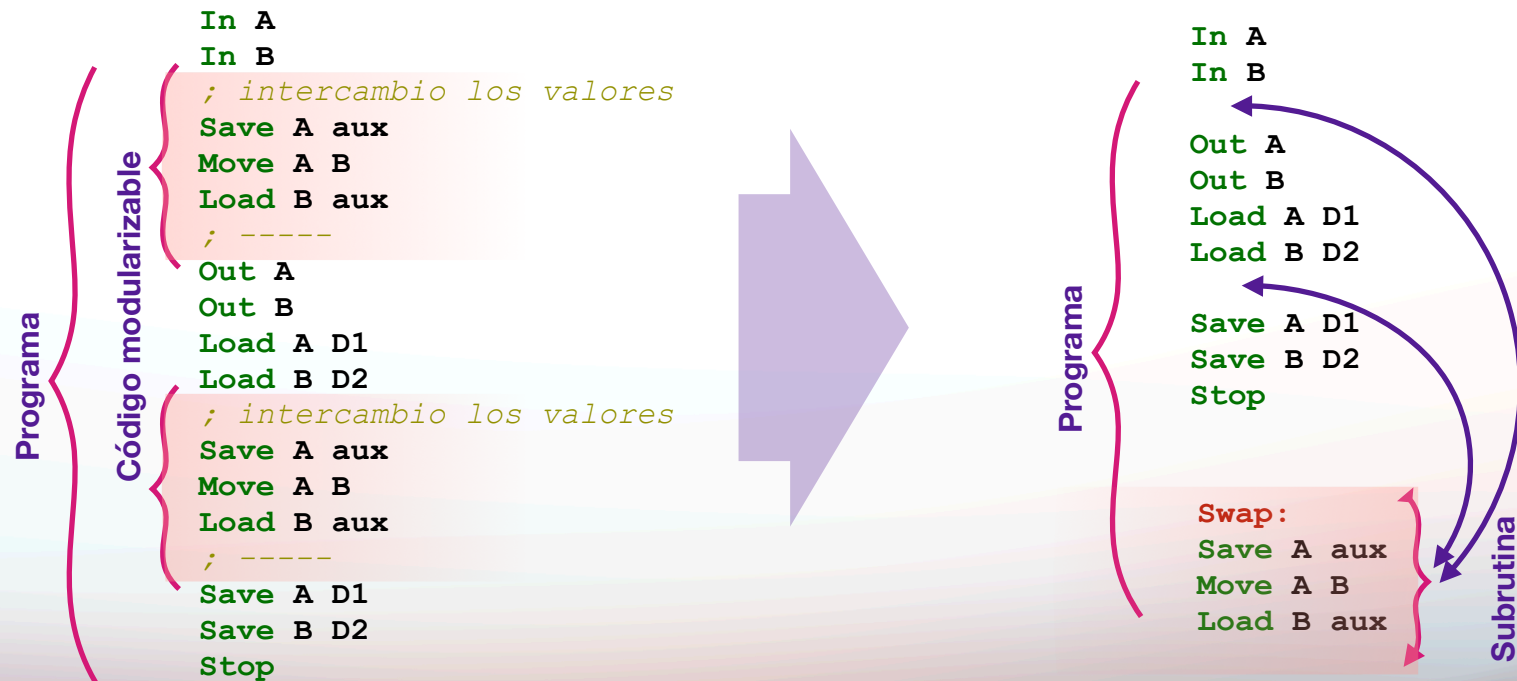
Una **subrutina** es una porción de código **independiente** que realiza una tarea específica dentro de un programa más grande. Las subrutinas se utilizan para **modularizar** el código y mejorar la **organización**, permitiendo **reutilizar** funcionalidades en diferentes partes del programa.

The diagram illustrates code modularization using nested curly braces. A large purple brace on the left is labeled 'Programa'. Inside it, a smaller purple brace is labeled 'Código modularizable'. The code is organized into two identical blocks, each enclosed in a light pink rectangular box. The first block contains instructions for swapping values of A and B, and the second block contains instructions for swapping values of A and B, and saving B to D2. The code is as follows:

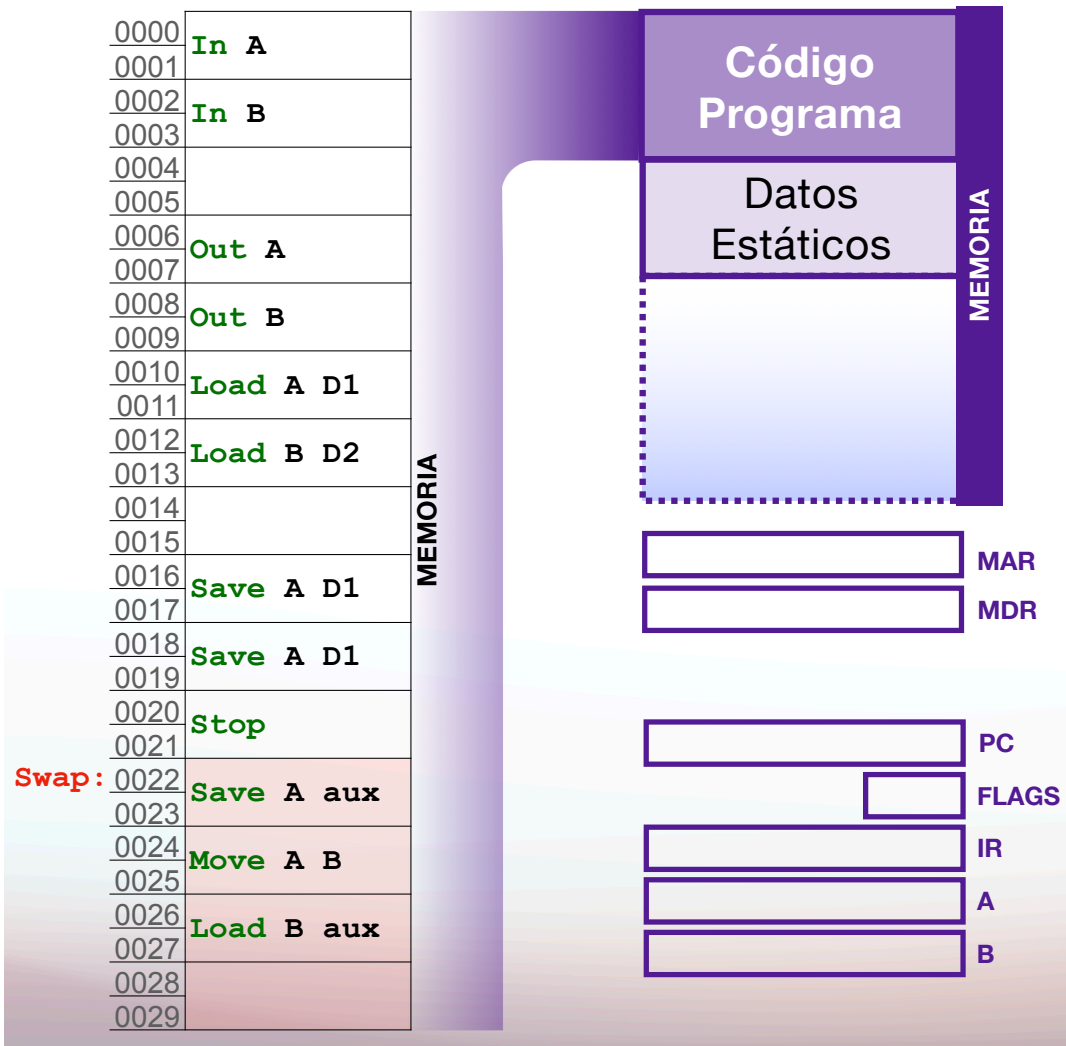
```
In A
In B
; intercambio los valores
Save A aux
Move A B
Load B aux
; -----
Out A
Out B
Load A D1
Load B D2
; intercambio los valores
Save A aux
Move A B
Load B aux
; -----
Save A D1
Save B D2
Stop
```

Hacia la posibilidad de modularizar código

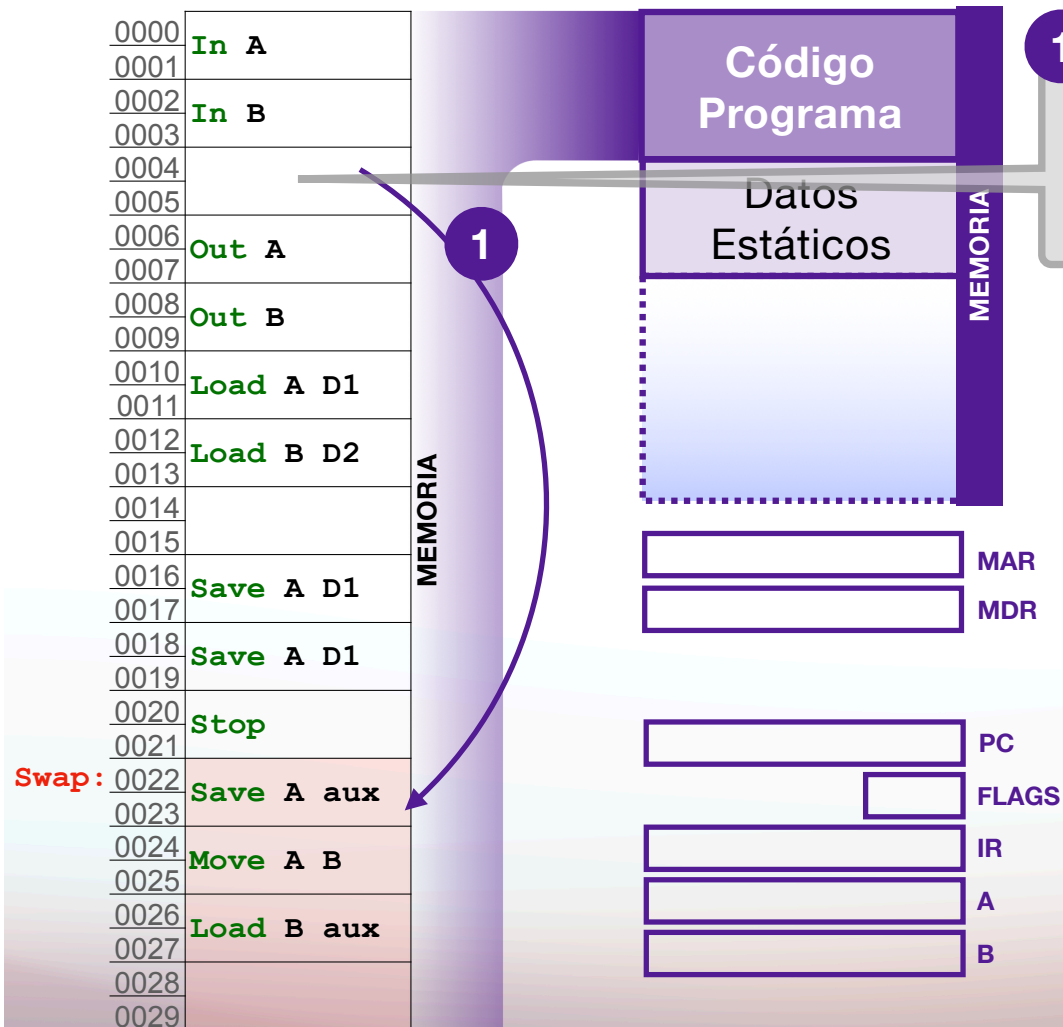
Una **subrutina** es una porción de código **independiente** que realiza una tarea específica dentro de un programa más grande. Las subrutinas se utilizan para **modularizar** el código y mejorar la **organización**, permitiendo **reutilizar** funcionalidades en diferentes partes del programa.



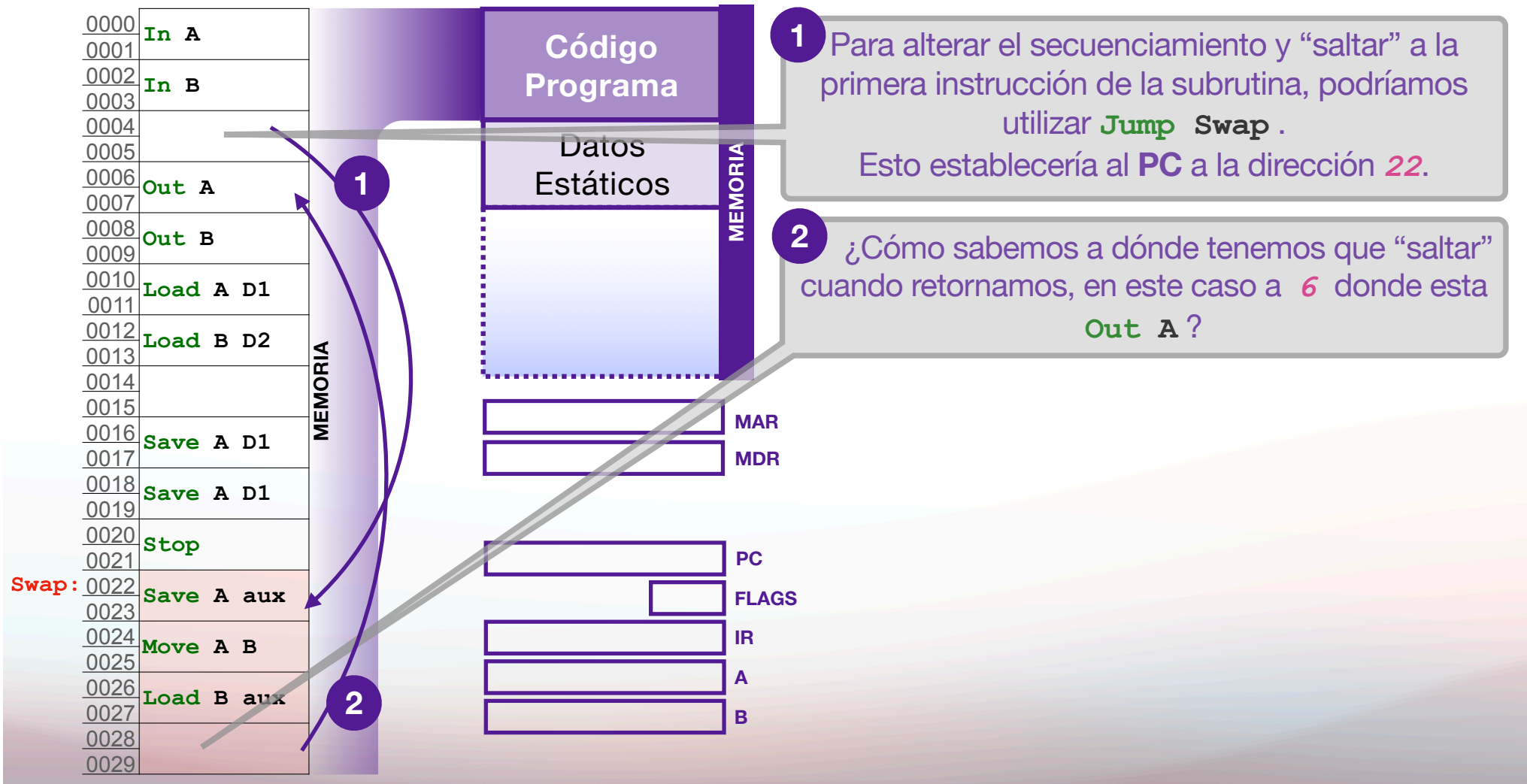
¿ cómo podemos implementar la idea ?



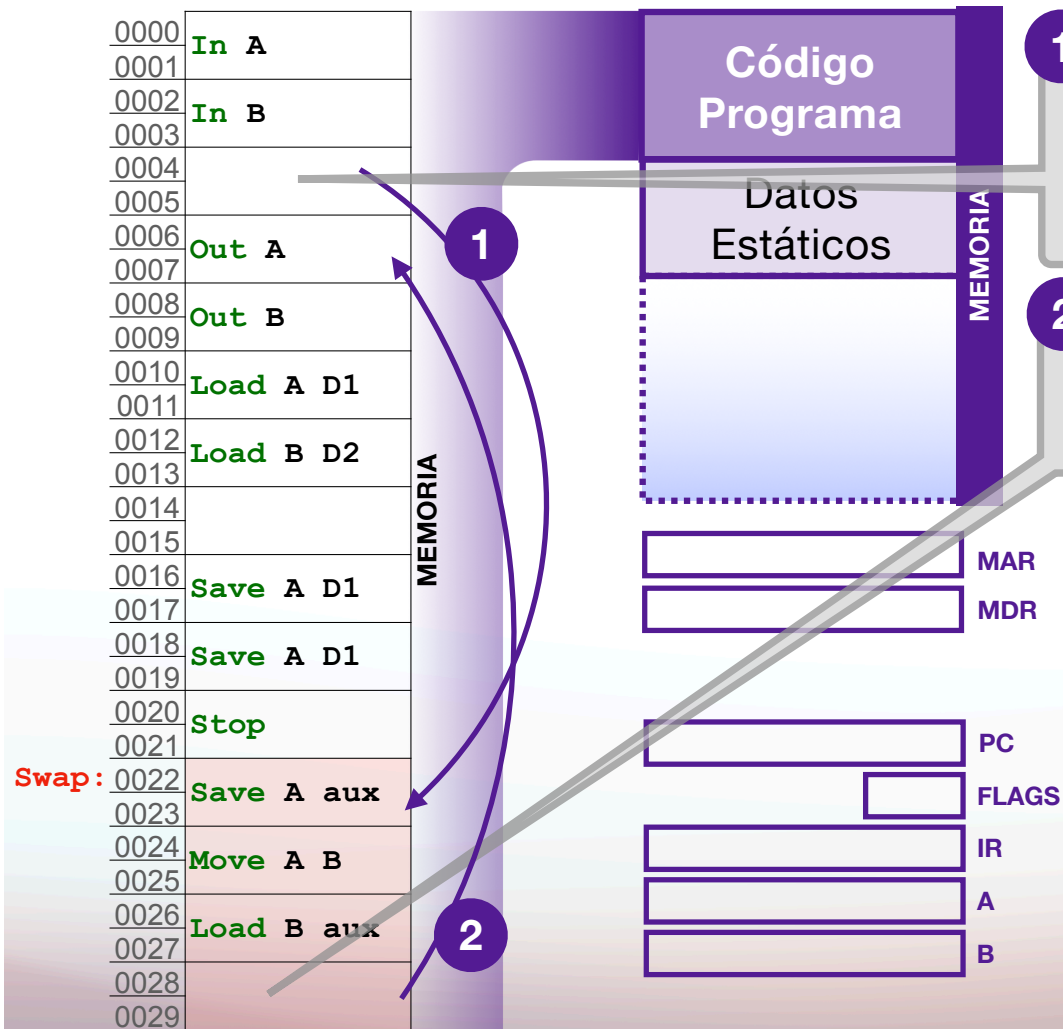
¿ cómo podemos implementar la idea ?



¿ cómo podemos implementar la idea ?



¿ cómo podemos implementar la idea ?



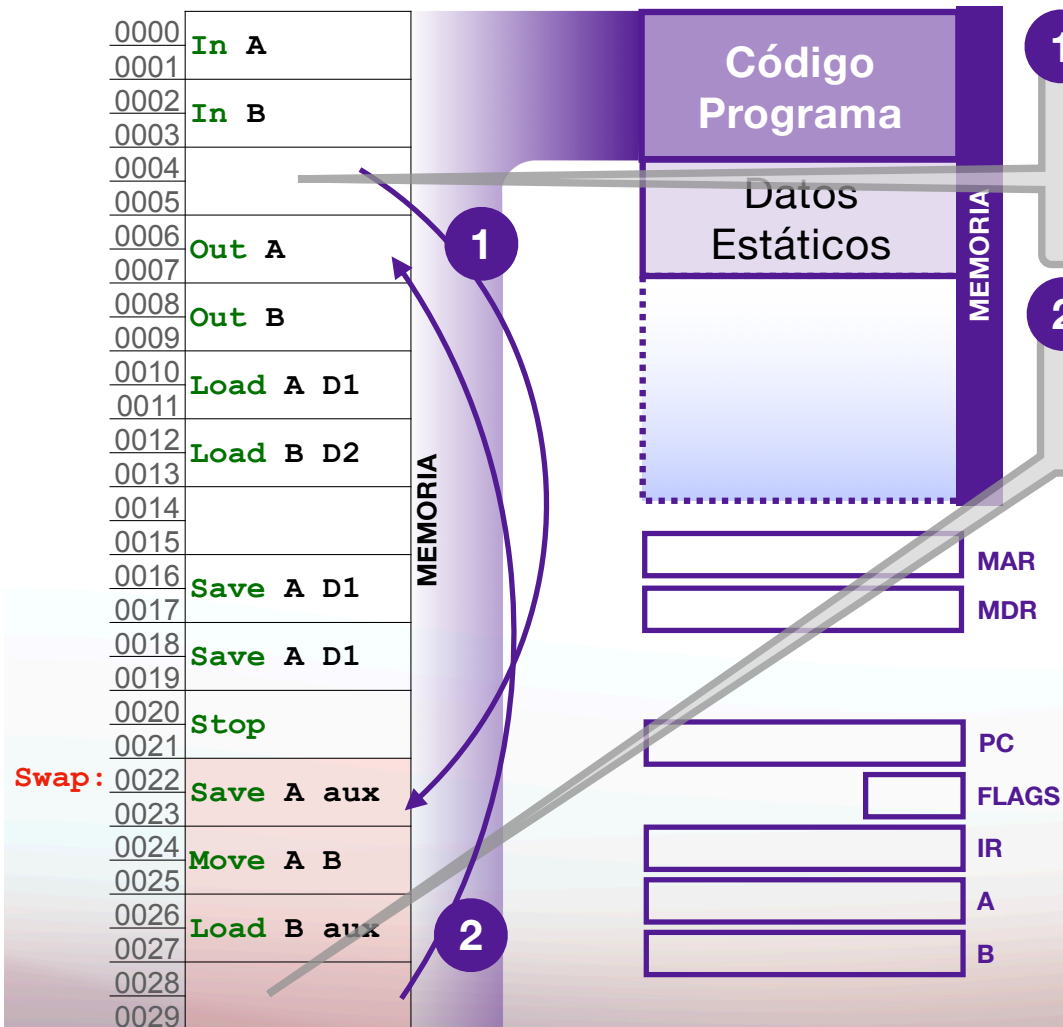
1 Para alterar el secuenciamiento y “saltar” a la primera instrucción de la subrutina, podríamos utilizar **Jump Swap** . Esto establecería al **PC** a la dirección **22**.

2 ¿Cómo sabemos a dónde tenemos que “saltar” cuando retornamos, en este caso a **6** donde esta **Out A** ?

Posibles soluciones:

- ◆ Podríamos cambiar código como **Jump 6**, pero ¿ qué sucede si la invoco desde diferentes puntos de mi programa?
- ◆ Podríamos guardar a dónde retornar (dirección de la próxima instrucción) en un registro, pero ¿ qué sucede si tenemos muchas invocaciones anidadas?, por ejemplo llamadas recursivas.

¿ cómo podemos implementar la idea ?



1 Para alterar el secuenciamiento y “saltar” a la primera instrucción de la subrutina, podríamos utilizar **Jump Swap** . Esto establecería al **PC** a la dirección **22**.

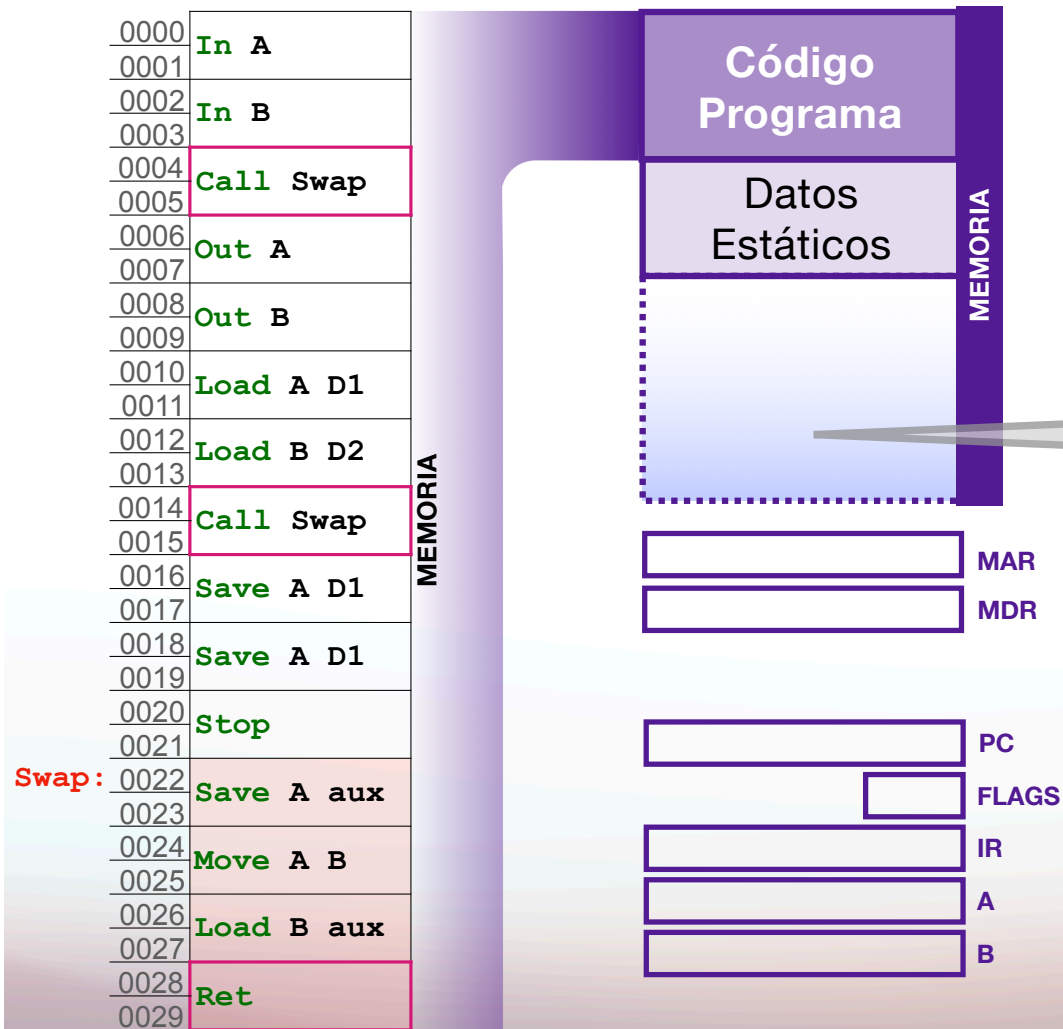
2 ¿Cómo sabemos a dónde tenemos que “saltar” cuando retornamos, en este caso a **6** donde esta **Out A** ?

Posibles soluciones:

- ◆ Podríamos cambiar código como **Jump 6**, pero ¿ qué sucede si la invoco desde diferentes puntos de mi programa?
- ◆ Podríamos guardar a dónde retornar (dirección de la próxima instrucción) en un registro, pero ¿ qué sucede si tenemos muchas invocaciones anidadas?, por ejemplo llamadas recursivas.

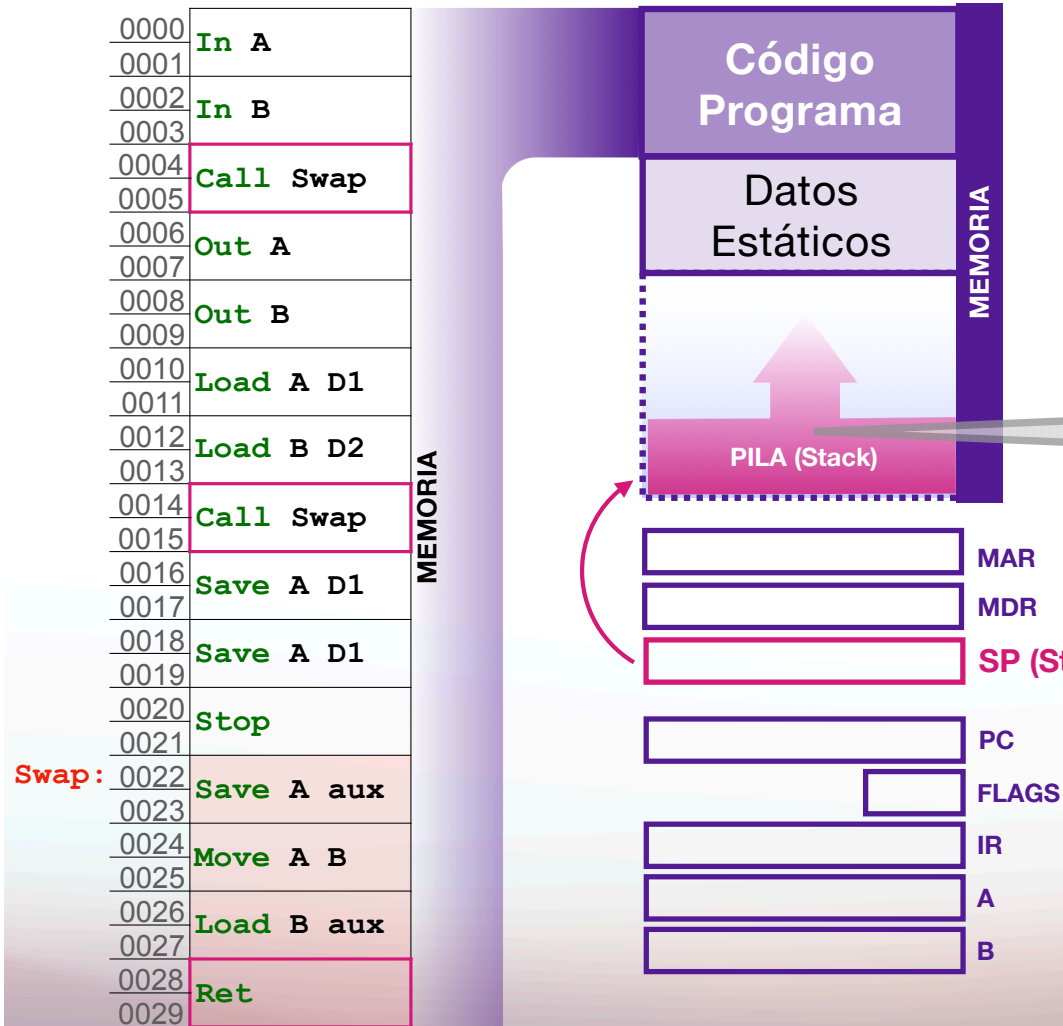
REJECT

La PILA (Stack), Call y Ret



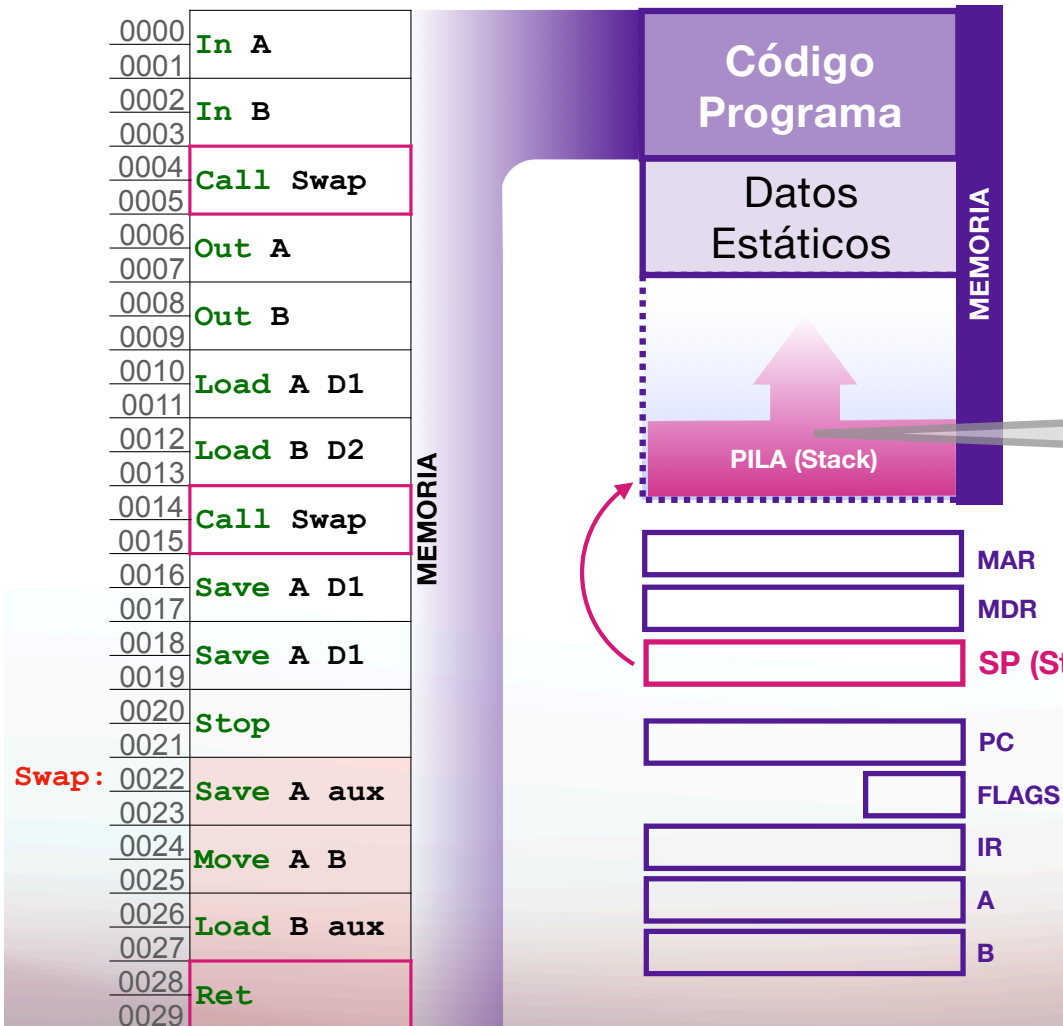
Para implementar la ejecución de subrutinas utilizamos una sección de la memoria denominada **PILA** (por la forma en que la usamos). Cuando invocamos a una subrutina **Call**, se **guarda** en la **PILA** cuál es la *dirección a la que tengo que retornar* y establecemos el **PC** a la dirección donde se encuentra la subrutina. Cuando finalizamos la ejecución de una subrutina **Ret**, retornamos (estableciendo el **PC**) a la dirección indicada en el **tope (SP)** de la PILA y la desapilamos.

La PILA (Stack), Call y Ret



Para implementar la ejecución de subrutinas utilizamos una sección de la memoria denominada **PILA** (por la forma en que la usamos). Cuando invocamos a una subrutina **Call**, se **guarda** en la **PILA** cuál es la *dirección a la que tengo que retornar* y establecemos el **PC** a la dirección donde se encuentra la subrutina. Cuando finalizamos la ejecución de una subrutina **Ret**, retornamos (estableciendo el **PC**) a la dirección indicada en el **tope (SP)** de la PILA y la desapilamos.

La PILA (Stack), Call y Ret



Para implementar la ejecución de subrutinas utilizamos una sección de la memoria denominada **PILA** (por la forma en que la usamos). Cuando invocamos a una subrutina **Call**, se **guarda** en la **PILA** cuál es la *dirección a la que tengo que retornar* y establecemos el **PC** a la dirección donde se encuentra la subrutina. Cuando finalizamos la ejecución de una subrutina **Ret**, retornamos (estableciendo el **PC**) a la dirección indicada en el **tope (SP)** de la PILA y la desapilamos.

Esto además resuelve el inconveniente con llamadas anidadas. Ya que las sucesivas direcciones de retorno se van **APILANDO** en el orden de invocación (anidamiento) y retornando en el sentido correcto.

Mejorando la Modularización (Parámetros)

Un aspecto crucial para la reutilización de subrutinas es la posibilidad de utilizar **parámetros**, lo que permite adaptar su comportamiento a diferentes contextos sin necesidad de modificar su implementación. ¿ cómo podemos implementarlo en bajo nivel ?. El uso de registros o espacios de memoria globales no resulta suficiente ni recomendable. La manera adecuada es a través del uso de la **PILA**.



Mejorando la Modularización (Parámetros)

Un aspecto crucial para la reutilización de subrutinas es la posibilidad de utilizar **parámetros**, lo que permite adaptar su comportamiento a diferentes contextos sin necesidad de modificar su implementación. ¿ cómo podemos implementarlo en bajo nivel ?. El uso de registros o espacios de memoria globales no resulta suficiente ni recomendable. La manera adecuada es a través del uso de la **PILA**.



Mejorando la Modularización (Parámetros)

Un aspecto crucial para la reutilización de subrutinas es la posibilidad de utilizar **parámetros**, lo que permite adaptar su comportamiento a diferentes contextos sin necesidad de modificar su implementación. ¿ cómo podemos implementarlo en bajo nivel ?. El uso de registros o espacios de memoria globales no resulta suficiente ni recomendable. La manera adecuada es a través del uso de la **PILA**.



Mejorando la Modularización (Parámetros)

Un aspecto crucial para la reutilización de subrutinas es la posibilidad de utilizar **parámetros**, lo que permite adaptar su comportamiento a diferentes contextos sin necesidad de modificar su implementación. ¿ cómo podemos implementarlo en bajo nivel ?. El uso de registros o espacios de memoria globales no resulta suficiente ni recomendable. La manera adecuada es a través del uso de la **PILA**.



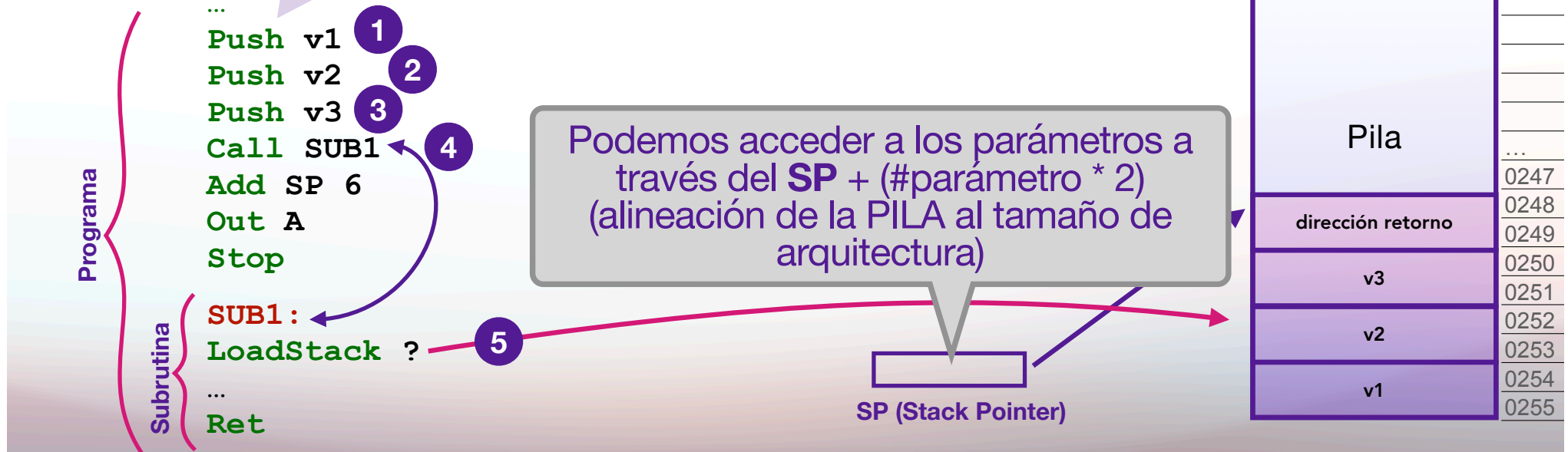
Mejorando la Modularización (Parámetros)

Un aspecto crucial para la reutilización de subrutinas es la posibilidad de utilizar **parámetros**, lo que permite adaptar su comportamiento a diferentes contextos sin necesidad de modificar su implementación. ¿ cómo podemos implementarlo en bajo nivel ?. El uso de registros o espacios de memoria globales no resulta suficiente ni recomendable. La manera adecuada es a través del uso de la **PILA**.



Mejorando la Modularización (Parámetros)

Un aspecto crucial para la reutilización de subrutinas es la posibilidad de utilizar **parámetros**, lo que permite adaptar su comportamiento a diferentes contextos sin necesidad de modificar su implementación. ¿ cómo podemos implementarlo en bajo nivel ?. El uso de registros o espacios de memoria globales no resulta suficiente ni recomendable. La manera adecuada es a través del uso de la **PILA**.



Mejorando la Modularización (Parámetros)

Un aspecto crucial para la reutilización de subrutinas es la posibilidad de utilizar **parámetros**, lo que permite adaptar su comportamiento a diferentes contextos sin necesidad de modificar su implementación. ¿ cómo podemos implementarlo en bajo nivel ?. El uso de registros o espacios de memoria globales no resulta suficiente ni recomendable. La manera adecuada es a través del uso de la **PILA**.



Mejorando la Modularización (Parámetros)

Un aspecto crucial para la reutilización de subrutinas es la posibilidad de utilizar **parámetros**, lo que permite adaptar su comportamiento a diferentes contextos sin necesidad de modificar su implementación. ¿ cómo podemos implementarlo en bajo nivel ?. El uso de registros o espacios de memoria globales no resulta suficiente ni recomendable. La manera adecuada es a través del uso de la **PILA**.



Mejorando la Modularización (Parámetros)

Un aspecto crucial para la reutilización de subrutinas es la posibilidad de utilizar **parámetros**, lo que permite adaptar su comportamiento a diferentes contextos sin necesidad de modificar su implementación. ¿ cómo podemos implementarlo en bajo nivel ?. El uso de registros o espacios de memoria globales no resulta suficiente ni recomendable. La manera adecuada es a través del uso de la **PILA**.



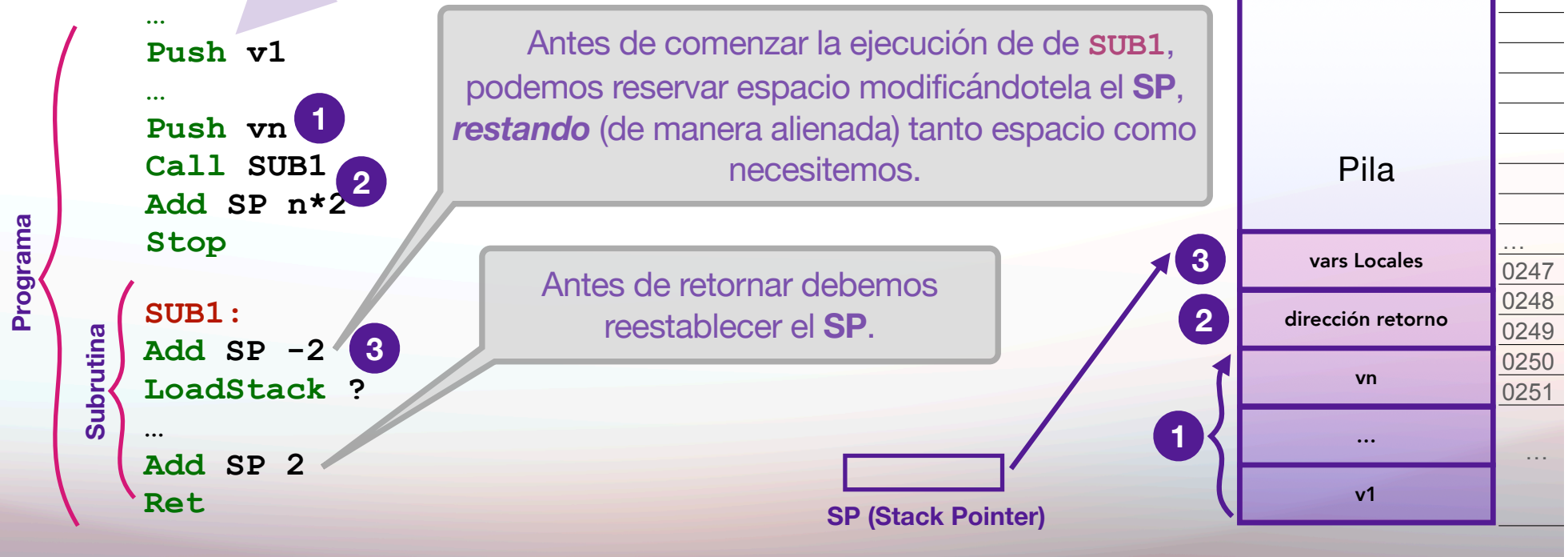
Mejorando la Modularización (Parámetros)

Un aspecto crucial para la reutilización de subrutinas es la posibilidad de utilizar **parámetros**, lo que permite adaptar su comportamiento a diferentes contextos sin necesidad de modificar su implementación. ¿ cómo podemos implementarlo en bajo nivel ?. El uso de registros o espacios de memoria globales no resulta suficiente ni recomendable. La manera adecuada es a través del uso de la **PILA**.



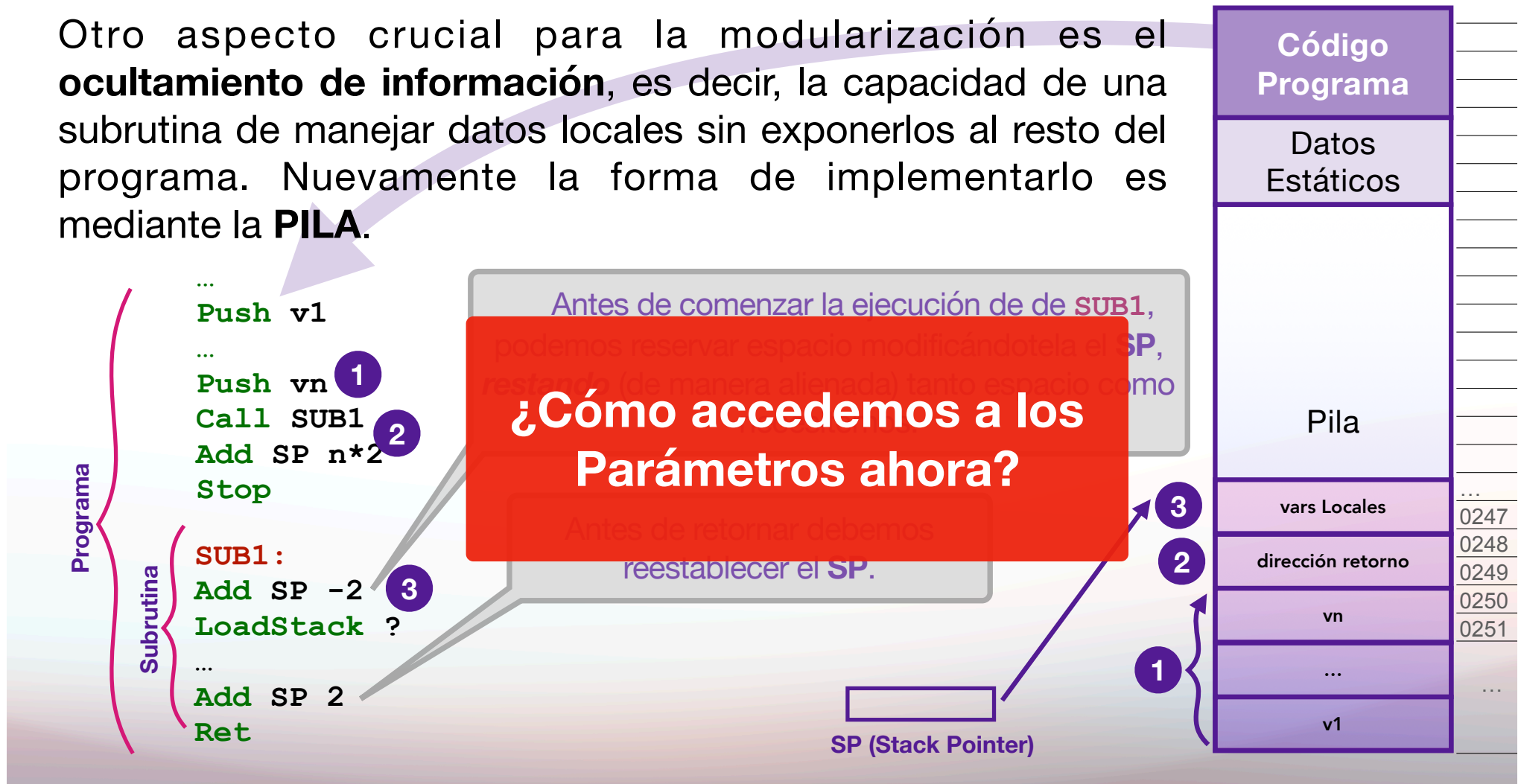
Mejorando la Modularización (Datos Locales)

Otro aspecto crucial para la modularización es el **ocultamiento de información**, es decir, la capacidad de una subrutina de manejar datos locales sin exponerlos al resto del programa. Nuevamente la forma de implementarlo es mediante la **PILA**.



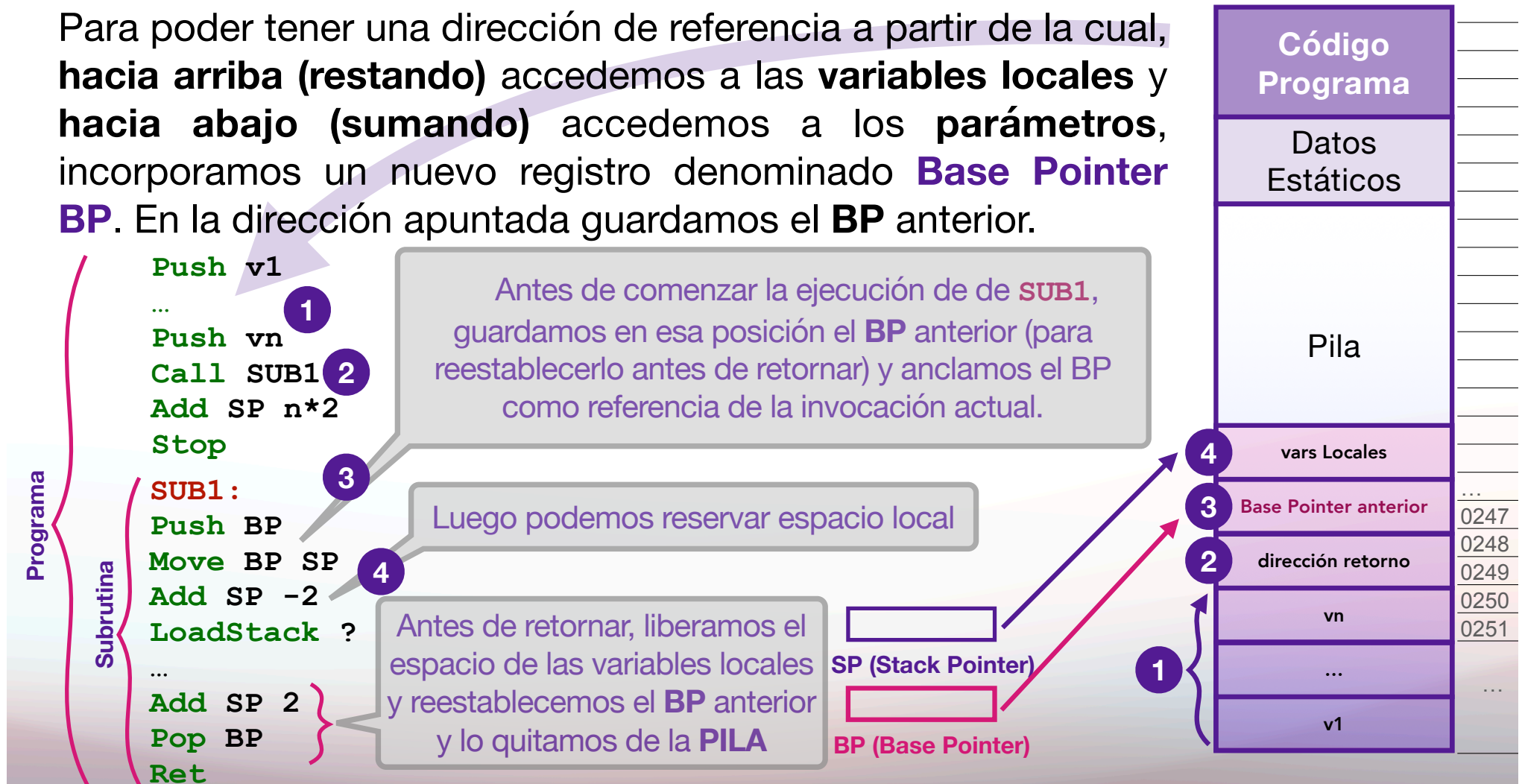
Mejorando la Modularización (Datos Locales)

Otro aspecto crucial para la modularización es el **ocultamiento de información**, es decir, la capacidad de una subrutina de manejar datos locales sin exponerlos al resto del programa. Nuevamente la forma de implementarlo es mediante la **PILA**.



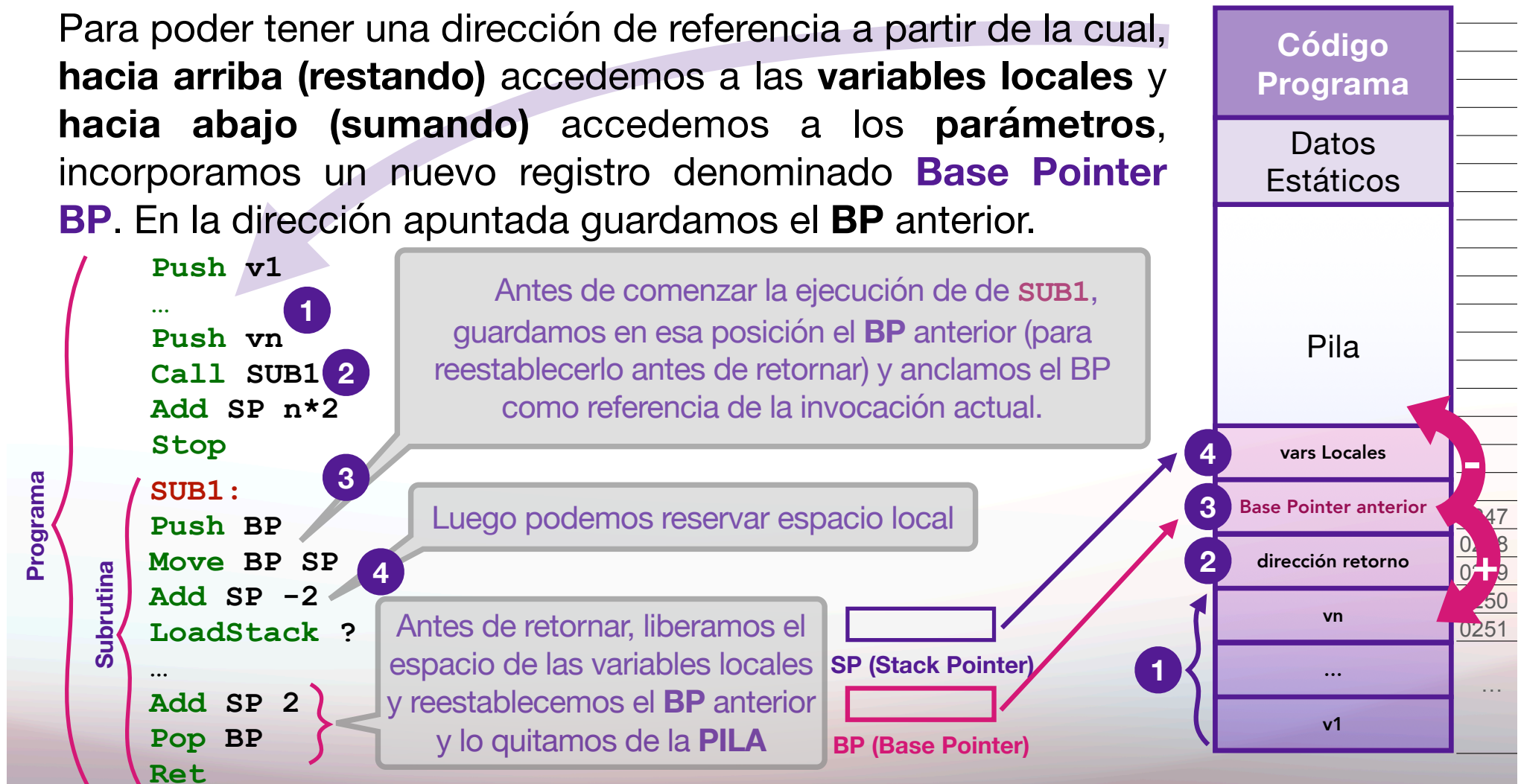
Mejorando la Modularización (Datos Locales)

Para poder tener una dirección de referencia a partir de la cual, **hacia arriba (restando)** accedemos a las **variables locales** y **hacia abajo (sumando)** accedemos a los **parámetros**, incorporamos un nuevo registro denominado **Base Pointer BP**. En la dirección apuntada guardamos el **BP** anterior.



Mejorando la Modularización (Datos Locales)

Para poder tener una dirección de referencia a partir de la cual, **hacia arriba (restando)** accedemos a las **variables locales** y **hacia abajo (sumando)** accedemos a los **parámetros**, incorporamos un nuevo registro denominado **Base Pointer BP**. En la dirección apuntada guardamos el **BP** anterior.



Subrutinas y su uso

Push v1

...

Push vn

Call SUB1

Add SP n*2

...

Stop

El invocante “**pushea**” en la PILA los **parámetros** a ser utilizados por la Subrutina (si los tiene)

El invocante “**invoca**” la Subrutina

El invocante “**quita**” los **parámetros** de la Subrutina

SUB1 :

Push BP

Move BP SP

Add SP -2

...

LoadStack ?

...

Add SP 2

Pop BP

Ret

Al comienzo de una Subrutina (**PROLOGO**), se debe:

-) **Guardar** el **Base Pointer** anterior en la pila
-) **Establecer** el **Base Pointer** para esta ejecución de la Subrutina
-) **Reservar** espacio para **variables locales** (si las necesita)

Se puede **acceder** a la pila utilizando **direcciones relativas al Base Pointer**

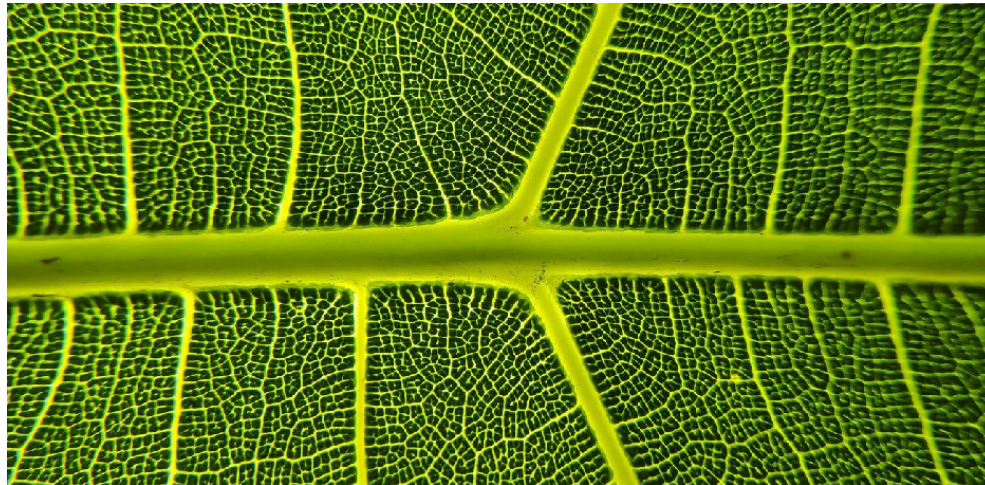
Si se reservó espacio para **variables locales**, hay que **reestablecerlo**

Antes de Retornar, **reestablecer el Base Pointer** anterior

Programa

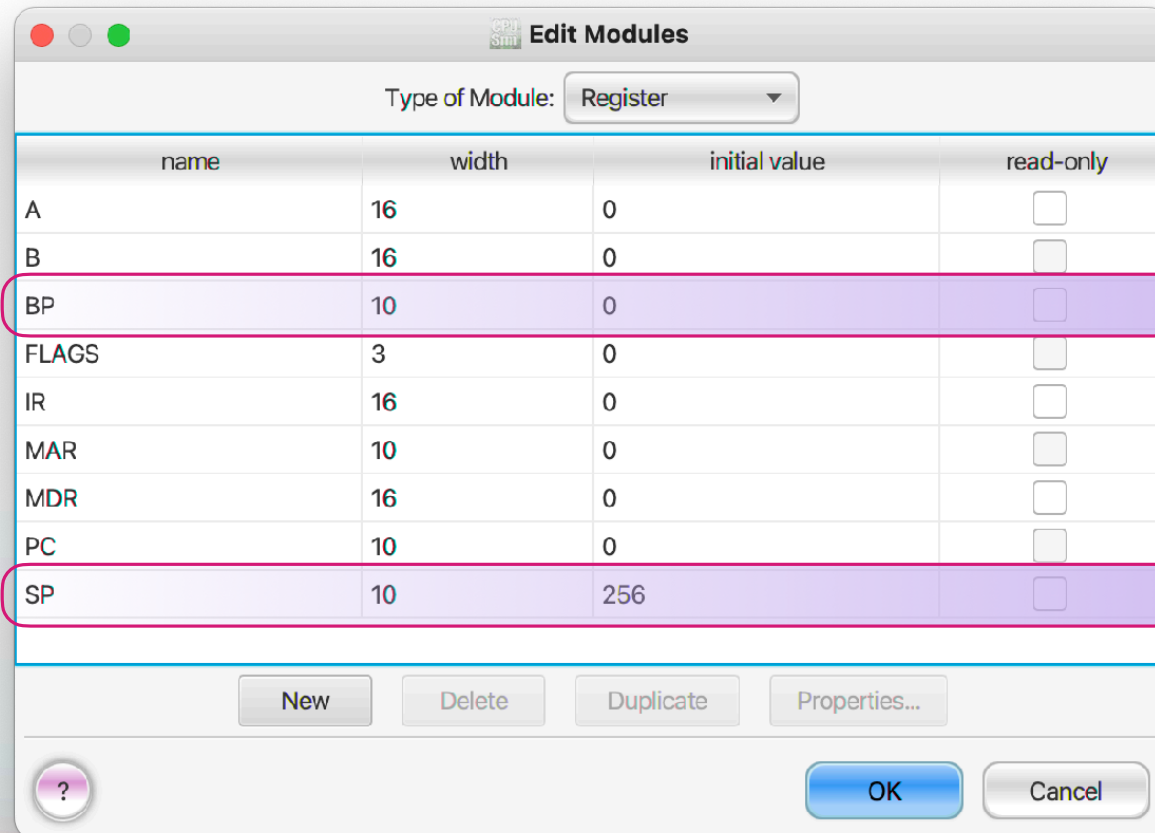
Subrutina

Hacia la posibilidad de modularizar código (ASSEMBLY)



Implementando en CPU-Sim nuestro procesador para soportar su utilización.

CPUSim - P1 (v2) Implementado Subrutinas - Hardware



CPUSim Edit Modules

Type of Module: Register

name	width	initial value	read-only
A	16	0	☐
B	16	0	☐
BP	10	0	☐
FLAGS	3	0	☐
IR	16	0	☐
MAR	10	0	☐
MDR	16	0	☐
PC	10	0	☐
SP	10	256	☐

New Delete Duplicate Properties...

? OK Cancel

Creamos el **Base Pointer (BP)**

Creamos el **Stack Pointer (SP)** y lo inicializamos por defecto con la dirección más grande de la **Mem**

CPUSim - P1 (v2) Implementado Subrutinas - Call

Call

Format Implementation

Instruction

Length: 16 Opcode 0x8

4 10 2

OPCODE DIRECCION NOUS...

Dirección de donde comienza la Subrutina

Format Implementation

Execute sequence

RESET-MDR-0

PC->MDR

DEC-SP

SP->MAR

MDR->Mem[MAR]

IR[4..13]->PC

End

Como debemos guardar la **dirección de retorno** (10 bits), alineado a la **PILA** (16 bits), debemos rellenar correctamente los bits restantes

Como el **PC** ya fue actualizado por el **FETCH**, mantiene la **dirección de retorno** que debemos guardar en la PILA. La copiamos al **MDR** para luego ser guardada en **Mem**

Decrementamos el **SP** (en 2) para reservar el lugar para guardar la **dirección de retorno** (movemos el tope).

Indicamos en el **MAR** la dirección del tope de la PILA (**SP**).

Guardamos en la PILA la **dirección de Retorno**

Modificamos el **Program Counter (PC)** con la dirección donde se encuentra la Subrutina (indicamos un **salto** a su comienzo).

Guardar la Dir. de Retorno

CPUSim - P1 (v2) Implementado Subrutinas - Ret

Ret

Format Implementation

Instruction

Length: 16 Opcode 0x9

4 12

OPCODE NOUSADO12

NO necesitamos ningún operando, ya que la dirección donde retornar está en el **tope de la PILA (SP)**

Recuperar y limpiar la Dir. de Retorno

Format Implementation

Execute sequence

SP->MAR

Mem[MAR]->MDR

INC-SP

MDR->PC

End

Indicamos en el **MAR** la dirección del tope de la PILA (**SP**).

Leemos de la PILA la **dirección de Retorno** y la guardamos en **MDR**. Como ya está en 16 bits (alienada) no es necesario rellenar el **MDR**.

Incrementamos el **SP** (en 2) para liberar el lugar que ocupaba la **dirección de retorno** (movemos el tope).

Modificamos el **Program Counter (PC)** con la dirección de Retorno (indicamos un **salto** de regreso a su invocante).

CPUSim - P1 (v2) Implementado Subrutinas - Push (con elección de Registro)

Push

Format Implementation

Instruction

Length: 16 Opcode 0xB

4 2 10

OPCODE REGID2 NOUSADO10

Incorporamos como operando, de qué registro toma el valor a “pushear”, recordemos que debemos tener en cuenta que puede ser el Base Pointer. Esto nos obliga a considerar 2 bits: (A=0, B=1, BP=2)

Execute sequence

DEC-SP

Decrementamos el **SP** (en 2) para reservar el lugar para guardar la dirección de retorno (movemos el tope).

SP->MAR

Indicamos en el **MAR** la dirección del tope de la PILA (**SP**).

A->MDR

<IR[4..5]==A>(4)

B->MDR

<IR[4..5]==B>(2)

Dependiendo del operando (REGID) guardo en **MDR** el valor del registro indicado, para luego ser guardado en el tope de la **PILA**. Movemos **A** y si era el indicado omitimos 4 microinstrucciones, sino movemos **B** y si era el indicado omitimos 2 microinstrucciones, sino completamos **MDR** con ceros y luego pasamos el valor del **BP** (10 bits)

RESET-MDR-0

BP->MDR

Guardamos en la PILA la el valor correspondiente

MDR->Mem[MAR]

End

CPUSim - P1 (v2) Implementado Subrutinas - Pop (con elección de Registro)

Pop

Format Implementation

Instruction

Length: 16 Opcode 0xC

4	2	10
OPCODE	REGID2	NOUSADO10

Incorporamos como operando, a qué registro transfiere el valor a “popear”. , recordemos que debemos tener en cuenta que puede ser el Base Pointer. Esto nos obliga a considerar 2 bits: (A=0, B=1, BP=2)

Execute sequence

SP->MAR

Indicamos en el **MAR** la dirección del tope de la PILA (**SP**).

Mem[MAR]->MDR

Leemos de la memoria el valor del tope de la PILA.

INC-SP

Incrementamos el **SP** (en 2) para liberar el lugar que ocupaba el valor en la PILA (movemos el tope).

<IR[4..5]==A>(5)

<IR[4..5]==B>(2)

MDR->BP

End

MDR->B

Dependiendo del operando (REGID) guardo el valor leído desde la PILA al **MDR** en el registro correspondiente.

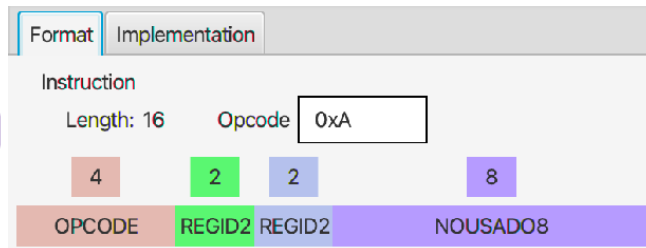
End

MDR->A

End

CPUSim - P1 (v2) Implementado Subrutinas - Move (con soporte para BP)

Move



Incorporamos como operando, a qué registro transfiere el valor a “popear”. , recordemos que debemos tener en cuenta que puede ser el Base Pointer. Esto nos obliga a considerar 2 bits: (A=0, B=1, BP=2).

Instrucciones soportadas:

Move A B ;mueve el contenido del Registro B al A

Move B A ;mueve el contenido del Registro A al B

Move BP SP ;mueve el contenido del Registro SP al BP

Execute sequence

Si es Reg A

<IR[4..5]!=BP>(2)

SP->BP

End

<IR[4..5]==B>(2)

B->A

End

A->B

End

Si no es **BP** omito 2 microinstrucciones

Muevo de **SP** a **BP** (ambos con 10 bits)

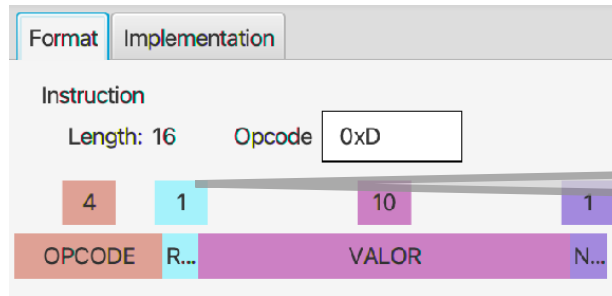
Si es **B** omito 2 microinstrucciones

Muevo de **B** a **A** (ambos con 16 bits)

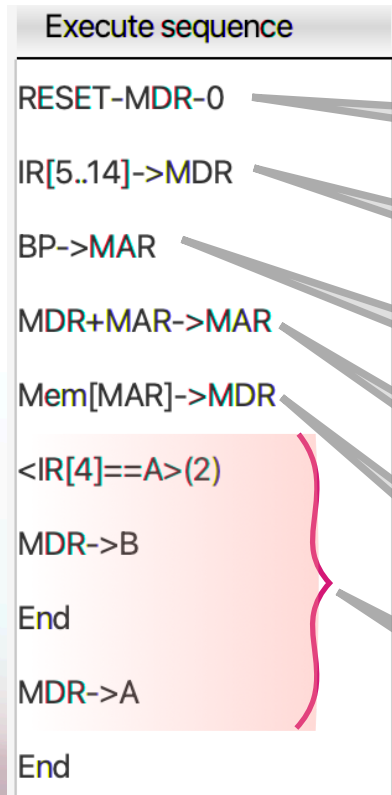
Muevo de **A** a **B** (ambos con 16 bits)

CPUSim - P1 (v2) Implementado Subrutinas - LoadStack

LoadStack



Incorporamos como operando, a qué registro transfiere el valor leído desde la PILA. Sólo se admiten los registros de Propósitos Generales A y B.



Dado que el valor es de 10 bits (inmediato), completamos el MDR con 0

Obtenemos de la Instrucción el valor a desplazar y lo almacenamos en MDR

Tomamos el valor del Base Pointer como Base

Calculamos el desplazamiento

Obtenemos el valor de la memoria

Dependiendo del registro destino copio el valor a registro correspondiente

CPUSim - P1 (v2) Usando Subrutinas - Desenmascarando la Recursión ;)

```
3 Main:
4     In A      ;leo de consola un valor en A
5     Push A;   ;cargo el parametro para Fact
6     Call Fact ; ;Invoco a Fact
7     Pop A ;   ;guardo parametro con el resultado en A
8     Out A ;   ;muestro el valor
9     Stop
10 Fact:
11     ;---- prologo
12     Push BP ;Push BP anterior
13     Move BP SP ;anclar BP a la invocacion actual
14     ;-----
15     LoadStack A 4 ;leo el parametro en A
16     Add A -1 I ;le resto 1
17     JumpZ end ; si es 0 entonces finalizo (caso base)
18     Push A ; si no es caso base cargo el parametro n-1
19     Call Fact ;invoco a fact
20     Pop A ; recupero el resultado parcial que dejo en parametro
21     LoadStack B 4 ;leo el n de la invocacion actual en B
22     Mult ; A*B ->A Fact(n-1)*n
23     SaveStack A 4; guardo el fact calculado actual en el paramtro
24 end:
25     Pop BP ;repago el BP anterior
26     Ret ;retorno
```